ANÁLISIS DE UN PROGRAMA CONCURRENTE GENERADO CON IA: BANCO CON TRANSFERENCIAS Y MÉTRICAS DE EVALUACIÓN

Integrantes:
Alan Nahuel GIARDELLI
agiardelli@alumno.unlam.edu.ar
Luis Alberto SAINI
lsaini@alumno.unlam.edu.ar
Alejandro RUIZ
alejaruiz@alumno.unlam.edu.ar
Ernesto OJEDA
ernojeda@alumno.unlam.edu.ar
Nazareno ORELLANO
nazorellano@alumno.unlam.edu.ar
Mariano RODRIGUEZ BUSTOS
mrodriguezbustos@alumno.unlam.edu.ar

Asignatura:
(01-3900) PROGRAMACIÓN CONCURRENTE

Fecha - Versión:
17/10/2026 – Versión 1.04

Resumen:

Este trabajo documenta la generación, mediante un asistente de IA (Claude, modelo Sonnet), de un programa concurrente en Java que simula un banco con transferencias entre cuentas, y su evaluación crítica. Se diseñó un prompt técnico, se analizaron los mecanismos de sincronización empleados (locks por cuenta con orden global, cola productor-consumidor y auditor con foto consistente) y se aplicaron rúbricas de funcionalidad y de corrección en concurrencia, junto con métricas de uso de CPU, memoria y tokens. El programa compiló sin errores, conservó el invariante de dinero en todas las ejecuciones, incluso bajo alta contención, y un modo de control sin sincronización permitió comprobar que el auditor detecta las condiciones de carrera. La revisión crítica encontró además una lectura sin protección en el propio auditor, por lo que la corrección en concurrencia se calificó con reservas. Un segundo ciclo de refinamiento (v2) agregó pruebas automáticas, validación de parámetros y detección de deadlocks sin cambiar el rendimiento, y las mediciones permitieron descartar una debilidad que se había sobreestimado. Se discuten las limitaciones de la evidencia empírica y del uso de IA para programación concurrente.

Palabras Clave: *programación concurrente, generación de código con IA, deadlock, condición de carrera, métricas de software*

I. CONTEXTO

El trabajo se desarrolló en la Universidad Nacional de La Matanza (UNLaM) y corresponde a la Actividad Práctica Obligatoria N°1 de la asignatura Programación Concurrente (01-3900) (2.º cuatrimestre de 2026). El área de conocimiento es la ingeniería de software concurrente, y la temática es el uso de herramientas de generación automática de código a partir de prompts (VibeCoding) para producir programas concurrentes y evaluar su calidad de forma cuantitativa y cualitativa.

II. INTRODUCCIÓN

La programación concurrente permite que varias tareas progresen de manera solapada, pero introduce errores que no existen en el código secuencial y que son difíciles de reproducir, como las condiciones de carrera, los deadlocks y las violaciones de atomicidad [1]–[3]. Un estudio sobre errores reales de concurrencia en aplicaciones de gran tamaño encontró que una gran parte de los deadlocks y de las violaciones de atomicidad se corrigen modificando la forma de adquirir locks o la extensión de las secciones críticas [3], lo que sugiere que el diseño de la sincronización es el punto más delicado.

Los modelos de lenguaje entrenados sobre código resuelven problemas de programación con una tasa de acierto considerable [4], pero la calidad de su código no es uniforme. En seguridad, por ejemplo, una proporción relevante del código sugerido por Copilot en escenarios de riesgo contenía vulnerabilidades [5]; y en código paralelo, los modelos rinden peor que en código secuencial [6]. Además, los estudios de usabilidad muestran que los programadores tienen dificultades para comprender, depurar y validar el código generado [7]. Por eso es necesario evaluar críticamente lo que la IA produce, en lugar de aceptarlo porque compila.

El objetivo de este trabajo es (i) especificar un problema concurrente no trivial mediante un prompt técnico, (ii) obtener el código con una herramienta de IA, (iii) analizarlo críticamente y (iv) aplicar métricas: cumplimiento funcional, corrección en concurrencia, CPU, memoria y tokens. Como marco teórico se usan los cuatro criterios de la rúbrica de concurrencia: sincronización, ausencia de deadlocks, ausencia de condiciones de carrera y exclusión mutua. El deadlock requiere exclusión mutua, retención y espera, ausencia de expropiación y espera circular [8]; la solución clásica que se aplica aquí rompe la condición de espera circular imponiendo un orden total en la adquisición de recursos [8], [9].

III. MÉTODOS

A. Herramienta, modelo y metodología de prompt

Se utilizó Claude Code con el modelo Sonnet, para generar código en Java 21 (solo biblioteca estándar). La metodología fue un prompt técnico inicial estructurado en cuatro partes, seguido de refinamientos: objetivo funcional, requisitos de concurrencia, interfaz/parámetros y entregable. El texto completo se transcribe a continuación (también está en el archivo prompt_tecnico.md del repositorio). Se pidió explícitamente un lock por cuenta (no uno global), orden creciente de adquisición, liberación en finally, cola acotada con poison pill, y un modo --unsafe como grupo de control. Se hicieron dos refinamientos: (1) tras la primera ejecución se corrigió la codificación de la consola (UTF-8), porque las tildes del informe final se veían mal; (2) después de una nueva corrida, el grupo pidió refinar el código para mejorarlo, y las mejoras concretas se eligieron a partir de las debilidades del análisis crítico de la v1 (sección IV.C). Ambos prompts se transcriben a continuación.

Prompt técnico inicial:

```
Actuá como ingeniero de software senior especializado en programación concurrente.
Generá un programa completo, compilable y ejecutable en Java 21 (solo librería
estándar, sin dependencias externas) que simule un banco con transferencias
concurrentes entre cuentas.
1. Objetivo funcional
- El banco tiene N cuentas (por defecto 50), cada una con saldo inicial configurable
(por defecto 10.000,00). Los montos se manejan en centavos con long.
- Varios hilos productores generan solicitudes de transferencia aleatorias (cuenta
origen, cuenta destino, monto) y las colocan en una cola acotada.
- Varios hilos trabajadores (consumidores) toman solicitudes de la cola y ejecutan la
transferencia.
- Una transferencia se rechaza si origen == destino o si el saldo de origen es
insuficiente. Nunca puede quedar un saldo negativo.
- Un hilo auditor verifica periódicamente que la suma de todos los saldos sea igual a
la suma inicial (invariante de conservación del dinero) tomando una foto consistente
del banco.
- Un hilo monitor imprime cada segundo un panel por consola: transferencias
procesadas, aceptadas, rechazadas, throughput, tamaño de la cola, resultado de la
última auditoría.
- Al finalizar se imprime un informe: tiempo total, throughput, transferencias por
trabajador, suma final vs. inicial y veredicto OK/FALLA (código de salida 0/1).
2. Requisitos de concurrencia
- Sincronización con ReentrantLock por cuenta (no usar un lock global para las
transferencias).
- Prevención de deadlock: adquirir siempre los locks en orden creciente de id de
cuenta. El auditor debe respetar el mismo orden.
- Sin condiciones de carrera: todo acceso al saldo debe estar protegido; contadores
globales con AtomicLong.
- Productores-consumidores con ArrayBlockingQueue; terminación limpia con poison pill
(sin Thread.stop, sin busy-waiting).
- Liberar los locks siempre en finally.
- Cierre ordenado: esperar (join) a todos los hilos antes del informe final.
3. Interfaz y parámetros
Parámetros por línea de comandos (todos opcionales): --cuentas, --saldo, --
productores, --workers, --transferencias, --cola, --auditMs, --seed, --quiet.
Agregar --unsafe, un modo deliberadamente sin sincronización, solo para demostrar la
condición de carrera y validar que el auditor la detecta.
El panel por consola debe ser claro y legible.
4. Entregable
Un único archivo BancoConcurrente.java comentado, más las instrucciones de compilación
y ejecución.
```

Prompt del segundo refinamiento (v2):

```
Refinar el código para mejorarlo (v2), atacando las debilidades del análisis crítico
de la v1:
1. Validar los parámetros con mensajes claros y código de salida 2; separar run() de
main (sin System.exit dentro de la lógica) para poder probarlo.
2. Evitar falso compartido en los contadores por worker (relleno de 16 longs).
3. Agregar un watchdog con ThreadMXBean que detecte deadlocks reales en ejecución.
4. Medir la pausa del auditor (media y máxima).
5. Agregar opciones --queue (array|linked) y --fair para comparar cola y justicia de
locks.
6. Agregar pruebas automáticas sin dependencias externas (conservación, estrés,
control unsafe, validación).
```

B. Problema y diseño del código generado

El programa (BancoConcurrente.java, unas 240 líneas en la v1) simula un banco con N cuentas (50 por defecto). Productores generan solicitudes de transferencia y las colocan en una ArrayBlockingQueue; trabajadores las consumen y las ejecutan; un auditor verifica periódicamente

que la suma de saldos no cambie y que ningún saldo sea negativo; un monitor muestra un panel por consola cada segundo. Los mecanismos de concurrencia son:

TABLA I

MECANISMOS DE CONCURRENCIA DEL CÓDIGO GENERADO

Mecanismo	**Dónde se usa**	**Propiedad que garantiza**
ReentrantLock por cuenta	Banco.transferir()	Exclusión mutua sobre el saldo de cada cuenta
Orden creciente de id	Banco.transferir() y sumaConsistente()	Sin espera circular, por lo tanto sin deadlock
try/finally con unlock()	Todas las secciones críticas	Los locks se liberan aun ante excepciones
ArrayBlockingQueue acotada	Productores → trabajadores	Sincronización con espera pasiva (sin busy-wait) y contrapresión
Poison pill + join()	Cierre ordenado	Terminación limpia sin Thread.stop
AtomicLong	Contadores globales	Incrementos atómicos sin lock
ThreadMXBean.findDeadlockedThreads()	Monitor (v2)	Detección de deadlock real en ejecución
Contadores por worker con relleno	Workers (v2)	Evita falso compartido (false sharing)
Locks de todas las cuentas en orden	Auditor	Foto consistente del banco (invariante verificable)

Fragmento central, tal como lo generó la IA:

```
Cuenta primero = a.id < b.id ? a : b;
Cuenta segundo = a.id < b.id ? b : a;
primero.lock.lock();
try {
    segundo.lock.lock();
    try { return mover(a, b, t.monto()); }
    finally { segundo.lock.unlock(); }
} finally { primero.lock.unlock(); }
```

C. Pruebas de validación

Además de compilar y ejecutar el programa, se hicieron tres tipos de prueba: (1) ejecuciones de 300.000 y 8.000.000 de transferencias con el auditor activo; (2) un estrés de alta contención, con solo 2 cuentas, 16 trabajadores y 4 productores (2.000.000 de transferencias, 3 corridas), que es el escenario donde un orden de locks incorrecto produciría deadlock; y (3) tres corridas en modo --unsafe como grupo de control, para comprobar que el verificador de invariante puede detectar errores y que, por lo tanto, un resultado OK es informativo.

D. Refinamiento (v2)

Tras una nueva corrida se refinó el código. Las mejoras se seleccionaron a partir de las debilidades del análisis crítico de la v1 (sección IV.C) y se conservó la v1 para comparar:

- Validación de parámetros con mensajes claros y código de salida 2; separación de run() y main(), para poder probar el programa sin System.exit.
- Contadores por worker con relleno de 128 bytes, para evitar falso compartido.
- Watchdog con ThreadMXBean que detecta deadlocks reales durante la ejecución y los informa por consola. Limitación: si un deadlock llegara a producirse, los trabajadores no terminarían y el programa quedaría esperando en join(), por lo que el veredicto final no llegaría a imprimirse; en las pruebas automáticas ese caso queda cubierto por el tiempo límite de cada corrida.
- Métrica de pausa del auditor (media y máxima), para cuantificar cuánto detiene a los trabajadores.
- Opciones --queue (array o linked) y --fair, para comparar el tipo de cola y la justicia de los locks con mediciones.
- Diez pruebas automáticas (PruebasBanco.java, sin dependencias externas): conservación del dinero, consistencia de contadores, alta contención, locks justos, cola enlazada, cero transferencias, saldos chicos, control negativo --unsafe y rechazo de parámetros inválidos.

E. Métricas y justificación de cada mecanismo de medición

Las métricas se registraron en la planilla Plantilla-metrica_completa.xlsx: la fila M1 corresponde a la v1 y la fila M2 a la v2.

Funcionalidad y concurrencia. El grupo aplicó las rúbricas de la cátedra, asignando a cada criterio el nivel A, B o C cuyo descriptor coincide con la evidencia observada; en la planilla, A, B y C se registran como Alta, Media y Baja. El criterio «funciona sin errores» y el «cumplimiento del objetivo funcional» se evaluaron compilando y comparando el comportamiento con cada requisito del prompt; el «diseño de la app» se evaluó sobre la interfaz de consola, según el criterio de la rúbrica (interfaz clara y fácil de usar). Esta métrica corresponde a la adecuación funcional (functional suitability) del modelo de calidad ISO/IEC 25010 [10]. Los cuatro criterios de concurrencia se evaluaron mediante revisión del código y las pruebas de la sección C. Se eligió este método porque la corrección de un programa concurrente no puede demostrarse solo ejecutándolo: una ejecución sin fallas no prueba ausencia de errores [3], de modo que se combinó el razonamiento sobre el diseño con pruebas de estrés y un grupo de control.

CPU y memoria. Se usó psutil [11], que muestrea desde otro proceso el consumo del proceso Java cada 100 ms. Se eligió psutil y no mediciones internas de la JVM porque mide el proceso completo, incluyendo el heap, los metadatos y el compilador JIT, tal como lo vería el sistema operativo; y porque es multiplataforma y reproducible con el script medir.py. La CPU se expresa en modo «100% = 1 núcleo», de modo que un valor mayor a 100% indica uso de más de un núcleo y el máximo posible es 100% × CPUs lógicas. La memoria es el RSS pico (conjunto residente) en MB. Ambas métricas permiten valorar la eficiencia de recursos, que es uno de los costos de la concurrencia: más hilos pueden mejorar el tiempo de respuesta, pero añaden cambios de contexto, pilas y contención. Cada acción se repitió 5 veces y se informa la media y la desviación estándar.

Tokens. La herramienta no entrega un contador directo en esta interfaz, por lo que se usó un cálculo matemático aproximado: unos 4 caracteres por token para el prompt en español (2.528 caracteres ≈ 632 tokens) y unos 3,5 caracteres por token para el código Java (11.231 caracteres ≈ 3.209 tokens), lo que da 3.841 tokens entre entrada y salida del código. Es una estimación y no incluye el contexto interno de la herramienta ni el razonamiento. Los tokens son relevantes porque determinan el costo y la latencia de la generación, y permiten comparar el esfuerzo de prompt con la calidad del resultado. Para la v2 se aplicó el mismo criterio al prompt de refinamiento (≈164 tokens) y al código nuevo (BancoConcurrente.java: 15.439 caracteres ≈ 4.411 tokens; PruebasBanco.java: 3.813 caracteres ≈ 1.090 tokens); el acumulado registrado en la planilla para M2 es de 9.507 tokens.

IV. RESULTADOS Y OBJETIVOS

A. Funcionalidad y concurrencia

TABLA II

EVALUACIÓN CON LAS RÚBRICAS DE LA CÁTEDRA: FUNCIONALIDAD 90/100 Y CONCURRENCIA 80/100 (V1 Y V2)

Rúbrica	Criterio	Nivel	Evidencia
Funcionalidad	Funciona sin errores	A (Alta)	Compila sin warnings; 15 corridas seguras sin excepciones
Funcionalidad	Cumplimiento del objetivo	A (Alta)	Se verificaron todos los requisitos del prompt (cola, locks, auditor, monitor, informe, parámetros)
Funcionalidad	Diseño de la app	B (Media)	Interfaz funcional con algunos problemas: el panel es legible, pero no hay ayuda de uso (--help se rechaza como parámetro desconocido) y los contadores del panel pueden mostrarse momentáneamente incoherentes
Concurrencia	Sincronización correcta	A (Alta)	Cola bloqueante, poison pill y join(); cierre limpio
Concurrencia	Ausencia de deadlocks	A (Alta)	No existen bloqueos y los recursos se liberan correctamente: orden global de locks, unlock en finally y 3 estreses con 2 cuentas y 16 hilos sin bloqueos
Concurrencia	Ausencia de condición de carrera	B (Media)	Saldo conservado en todas las corridas seguras y 3/3

			corridas unsafe fallan, pero el auditor lee los saldos sin locks en haySaldoNegativo() (data race formal)
Concurrencia	Exclusión mutua	B (Media)	Protección parcial o excesiva: lock por cuenta y unlock en finally, pero la lectura de haySaldoNegativo() queda fuera de la sección crítica (parcial) y el auditor bloquea todas las cuentas a la vez para cada foto (excesiva)

En el modo de control --unsafe, las tres corridas con 8 trabajadores y 300.000 transferencias terminaron con diferencias de +584,94, +542,28 y +446,20 respecto del saldo inicial (se creó dinero por actualizaciones perdidas), y el programa reportó FALLA. En el modo seguro, la diferencia fue 0,00 en todas las corridas (resultado que no descarta el defecto descrito en IV.C, porque la lectura sin protección no altera los saldos), y las 6 auditorías periódicas de la ejecución de 8.000.000 de transferencias dieron OK. Esto muestra que el verificador detecta carreras y que el código seguro las evita. La v2 obtiene los mismos niveles en las rúbricas (fila M2) y sus 10 pruebas automáticas pasan en alrededor de 1,5 s.

B. CPU, memoria y rendimiento

TABLA III

USO DE RECURSOS (5 REPETICIONES POR CELDA; CPU EN MODO 100% = 1 NÚCLEO; 2 CPUS LÓGICAS; 8.031 MB DE RAM). ACCIÓN 1: 4 WORKERS, 50 CUENTAS, 8M TRANSF. ACCIÓN 2: 16 WORKERS, 2 CUENTAS, 8M TRANSF.

Versión	**Acción 1: CPU % / tiempo (s)**	**Acción 2: CPU % / tiempo (s)**	**Memoria pico (MB) A1 / A2**
v1 (primera medición)	129,1 / 3,57	105,6 / 6,61	102,9 / 104,0
v1 (re-medida con v2)	128,1 ± 1,0 / 2,95	101,3 ± 1,4 / 6,93	103,2 / 104,2
v2 (cola array, por defecto)	127,2 ± 1,1 / 3,07	101,5 ± 0,8 / 6,57	103,9 / 104,7
v2 --queue linked	180,1 ± 3,4 / 4,33	181,0 ± 3,5 / 4,12	104,0 / 105,2
v2 --fair	129,2 ± 1,7 / 3,16	154,6 ± 1,5 / 21,19	104,2 / 105,9

Con 2 CPUs lógicas, el máximo teórico es 200%. En la acción 1 el programa usó en promedio 1,3 núcleos: una explicación probable, que no se midió, es que los hilos pasan tiempo bloqueados en la cola y por eso no saturan el procesador. En la acción 2 el uso baja a 1,0 núcleo y el tiempo se duplica con respecto a la acción 1 para la misma cantidad de transferencias (6,6 s frente a 3,1 s en la v2): con solo 2 cuentas, casi todas las transferencias compiten por los mismos dos locks, por lo que el trabajo se serializa. Es el comportamiento esperado de la ley de Amdahl: la parte serializada limita el aumento de velocidad. La memoria es casi constante (~103–105 MB), porque el programa tiene poco estado y el consumo está dominado por la JVM.

Entre v1 y v2 no hay diferencias de rendimiento apreciables (los refinamientos apuntaban a robustez y verificabilidad, no a velocidad). Entre dos sesiones de medición, el mismo código v1 varió alrededor de un 20% en tiempo (3,57 s y 2,95 s en la acción 1), por lo que el entorno virtualizado compartido introduce ruido y solo las comparaciones dentro de una misma sesión son confiables. Las variantes sí muestran compromisos: la cola enlazada (posiblemente por sus dos locks internos, algo que no se verificó) reduce el tiempo del estrés de 6,57 s a 4,12 s, pero empeora la simulación estándar de 3,07 s a 4,33 s, de modo que ninguna es mejor en todos los escenarios y se mantuvo la cola array por defecto. Los locks justos (FIFO estricto) multiplican por unas 3,2 veces el tiempo del estrés (21,19 s), un costo alto por una garantía que este programa no necesita para ser correcto.

Pausa del auditor. Para evaluar la debilidad señalada en la v1 (el auditor toma todos los locks), se midió el tiempo que tarda en obtenerlos y sumar, con 3 corridas de 8M transferencias por celda:

TABLA IV

COSTO DE LA FOTO CONSISTENTE DEL AUDITOR (V2)

Cuentas	**Período de auditoría**	**Throughput medio (transf./s)**	**Pausa media (ms)**	**Pausa máxima (ms)**
50	1000 ms	2.814.668	0,049	0,147
50	10 ms	2.791.717	0,014	1,278

5000	1000 ms	2.088.585	0,317	0,408
5000	10 ms	2.066.555	0,134	4,547

Aun auditando cada 10 ms, el throughput no cambia de forma visible, y las pausas medias son de décimas de milisegundo. La caída de throughput con 5.000 cuentas (alrededor de 2,1M frente a 2,8M transf./s) aparece igual con auditorías cada 10 ms y cada 1000 ms, por lo que no se debe al auditor (una explicación plausible, que no se verificó, es el menor aprovechamiento de la caché con más cuentas). Por lo tanto, la crítica inicial al auditor stop-the-world estaba sobreestimada para estas escalas. La medición se hizo en un entorno virtualizado con 2 CPUs lógicas, por lo que los valores absolutos pueden variar al repetir medir.py en otra máquina.

C. Análisis crítico del código

Fortalezas. La IA aplicó correctamente la técnica clásica de orden total de recursos, incluyó el unlock en finally, usó espera pasiva en lugar de busy-wait y pensó en la terminación (poison pill). También hizo que el auditor use el mismo orden de locks, lo cual es un punto fácil de olvidar y habría reintroducido el riesgo de deadlock. El código es claro, está comentado y se comporta como se pidió.

Defecto no corregido (hallado en la revisión posterior): en el auditor, tras liberar los locks, haySaldoNegativo() recorre los saldos sin ninguna sincronización mientras los trabajadores los modifican. Según el modelo de memoria de Java es una condición de carrera sobre datos; en la práctica es de bajo riesgo, porque el algoritmo nunca deja un saldo negativo, pero la protección no es completa y por eso se calificó B en los criterios de condiciones de carrera y exclusión mutua. La corrección es simple (hacer la verificación dentro de sumaConsistente(), con los locks tomados). Otro riesgo menor: los workers solo capturan InterruptedException; una excepción inesperada mataría al worker en silencio y, si murieran todos, los productores quedarían bloqueados. Por último, el panel del monitor lee los contadores (procesadas, aceptadas y rechazadas) en instantes distintos y sin una foto atómica, por lo que puede mostrar valores momentáneamente incoherentes: en una corrida de 8.000.000 de transferencias, al primer segundo informó 3.269.584 aceptadas y 146.200 rechazadas (3.415.784 en total), más que las 3.413.598 procesadas. No afecta los saldos ni el informe final, que se calcula después de join(), pero es una imprecisión de la interfaz.

Debilidades identificadas en la v1 y su estado en la v2:

- Auditor stop-the-world: se midió (Tabla IV) y la pausa es de décimas de milisegundo, sin efecto visible en el throughput; la debilidad era menor de lo supuesto a esta escala. Podría importar con muchas más cuentas o con mucha más contención, y la v2 deja la métrica de pausa incorporada.
- Cola con un único lock interno: se comparó con LinkedBlockingQueue (Tabla III); mejora el estrés pero empeora el caso estándar, de modo que es una decisión dependiente de la carga y no una mejora general.
- Falso compartido en los contadores por worker: la v2 agrega relleno; no se midió un cambio de rendimiento apreciable, pero se conserva como buena práctica. La escritura sin sincronización a porWorker sigue siendo correcta (un único escritor por posición, lectura tras join(), relación happens-before).
- Locks no justos: la v2 permite --fair; el costo es de unas 3,2 veces en alta contención. En las corridas estándar los contadores por worker fueron parejos (entre 1,98M y 2,06M), sin evidencia de inanición, aunque no hay garantía formal.
- Sin pruebas automáticas, validación mínima y System.exit en la lógica: resuelto en la v2 (10 pruebas, validación con código de salida 2, run() separado de main()).
- La evidencia de ausencia de deadlock y de carreras sigue siendo empírica: las pruebas y el watchdog aumentan la confianza y detectan fallas en ejecución, pero no demuestran corrección. La

garantía sólida proviene del razonamiento sobre el orden de adquisición de locks. Una herramienta específica de pruebas de concurrencia (por ejemplo jcstress) daría más evidencia y queda pendiente.

D. Objetivos futuros

Repetir las mediciones en las máquinas del grupo, incorporar herramientas específicas de prueba de concurrencia (por ejemplo, jcstress), comparar con otro modelo o con otra metodología de prompt (por ejemplo, prompts con ejemplos), y evaluar el auditor con muchas más cuentas.

V. CONCLUSIONES

La IA generó, a partir de un único prompt técnico detallado, un programa concurrente correcto en lo esencial: compiló, cumplió los requisitos funcionales y conservó el invariante de dinero en todas las pruebas, incluso con alta contención. La calidad del resultado dependió mucho de la especificación: cada mecanismo correcto (locks por cuenta, orden global, poison pill, auditor consistente) estaba pedido en el prompt, de modo que no puede afirmarse que la IA lo hubiera elegido sola. Esto es coherente con los hallazgos sobre la dependencia de la calidad respecto del prompt en código paralelo [6].

Entre las ventajas se cuentan la rapidez para obtener un esquema completo, legible y comentado, y la capacidad de incluir un modo de control útil para validar. Entre las limitaciones, la verificación de corrección sigue en manos de quien programa: la ausencia de fallas observadas no demuestra la ausencia de errores [3], y el código generado tiene decisiones de diseño (auditor stop-the-world, cola con un único lock) que solo se detectan con análisis crítico y mediciones. Por lo tanto, la IA es una herramienta de apoyo que acelera la escritura, pero no reemplaza el razonamiento sobre sincronización ni las pruebas.

El ciclo de refinamiento mostró que la IA puede mejorar la robustez y la verificabilidad del código (pruebas, validación, detección de deadlocks) sin alterar su comportamiento, y que medir permite descartar optimizaciones innecesarias: una de las debilidades señaladas en el análisis inicial, la pausa del auditor, resultó despreciable a esta escala, mientras que las dos variantes de rendimiento probadas (cola enlazada y locks justos) tienen compromisos que solo se ven al medir.

Limitaciones del estudio: la evaluación con las rúbricas la realizó el propio grupo y, aunque se apoyó en pruebas reproducibles, conserva un componente de juicio (una revisión por pares externa sería más objetiva); las pruebas de estrés se corrieron en solo 2 CPUs lógicas, que exponen menos intercalaciones que una máquina con más núcleos; un solo problema, un solo modelo y dos generaciones (v1 y v2), una estimación aproximada de tokens, y mediciones tomadas en un entorno de 2 CPUs lógicas; los resultados no deben generalizarse sin repetir el experimento.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas:

[1] E. W. Dijkstra, «Solution of a problem in concurrent programming control», Commun. ACM, vol. 8, n.º 9, p. 569, sep. 1965.

[2] C. A. R. Hoare, «Monitors: An operating system structuring concept», Commun. ACM, vol. 17, n.º 10, pp. 549–557, oct. 1974.

[3] S. Lu, S. Park, E. Seo y Y. Zhou, «Learning from mistakes: A comprehensive study on real world concurrency bug characteristics», en Proc. 13th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2008, pp. 329–339.

[4] M. Chen et al., «Evaluating large language models trained on code», arXiv:2107.03374, jul. 2021.
[5] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt y R. Karri, «Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions», en Proc. IEEE Symp. Security and Privacy (SP), 2022, pp. 754–768.
[6] D. Nichols et al., «Can large language models write parallel code?», en Proc. 33rd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC), 2024.
[7] P. Vaithilingam, T. Zhang y E. L. Glassman, «Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models», en Extended Abstracts of the CHI Conf. Human Factors in Computing Systems, 2022.
[8] E. G. Coffman, M. Elphick y A. Shoshani, «System deadlocks», ACM Comput. Surv., vol. 3, n.º 2, pp. 67–78, jun. 1971.
[9] M. Herlihy y N. Shavit, The Art of Multiprocessor Programming, ed. rev. Waltham, MA, EE. UU.: Morgan Kaufmann, 2012.
[10] ISO/IEC, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, ISO/IEC 25010:2011, 2011.
[11] G. Rodolà, «psutil documentation» [en línea]. Consultado: 17 oct. 2026. Disponible: https://psutil.readthedocs.io

B. Bibliografía:

B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes y D. Lea, Java Concurrency in Practice. Upper Saddle River, NJ, EE. UU.: Addison-Wesley, 2006.